OxyGen: Unified KV Cache Management for VLA Inference under Multi-Task Parallelism

Xuangyu Li, Huaizhi Tang, Xin Ding, Weijun Wang, Ting Cao, Yunxin Liu

ArXiv 2026-5-19

Presenter: Seonghun Jung

KAIST

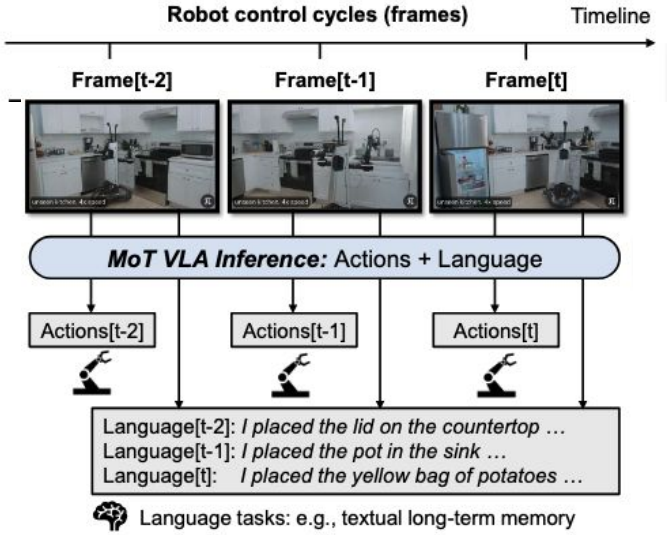
CASYS | KAIST Computer Architecture & System Lab

Contents

- **Introduction**
  - Background
  - Motivation
- **Approach**
  - Key Idea
  - Architecture
- **Evaluation**
  - Methodology
  - Results
- **Conclusion**

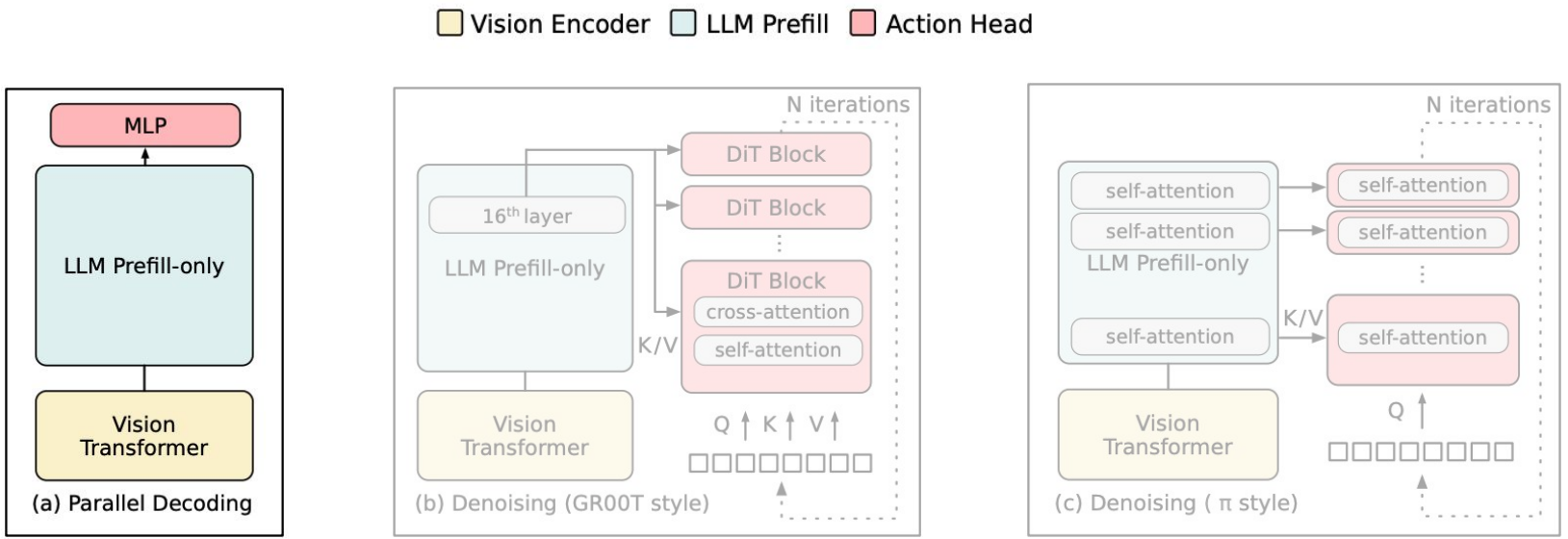
Multi-Task Embodiment

- **Embodied agents must run several tasks in parallel from one shared observation**
  - Manipulate while narrating progress, building memory, and re-planning.
  - Tasks are temporally independent but consume the same input with different timing constraint



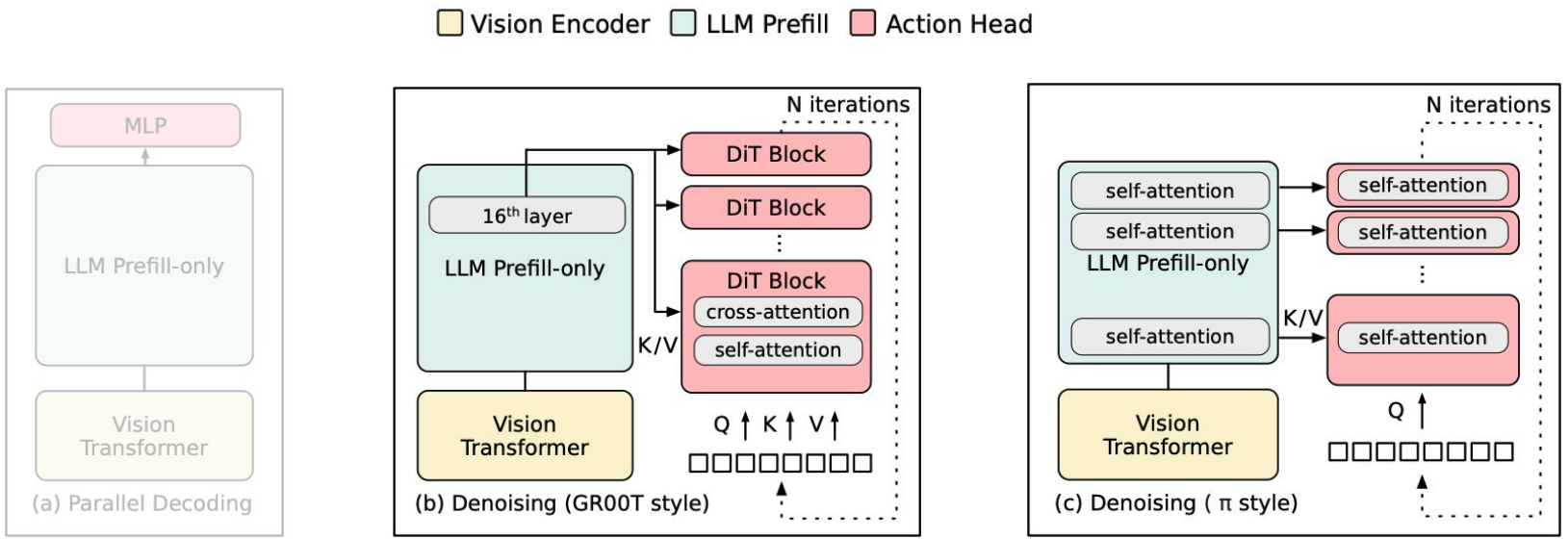
Conventional VLAs

- **Single modality pipeline**
  - A second task (narration, planning, memory) means loading and running a second model.



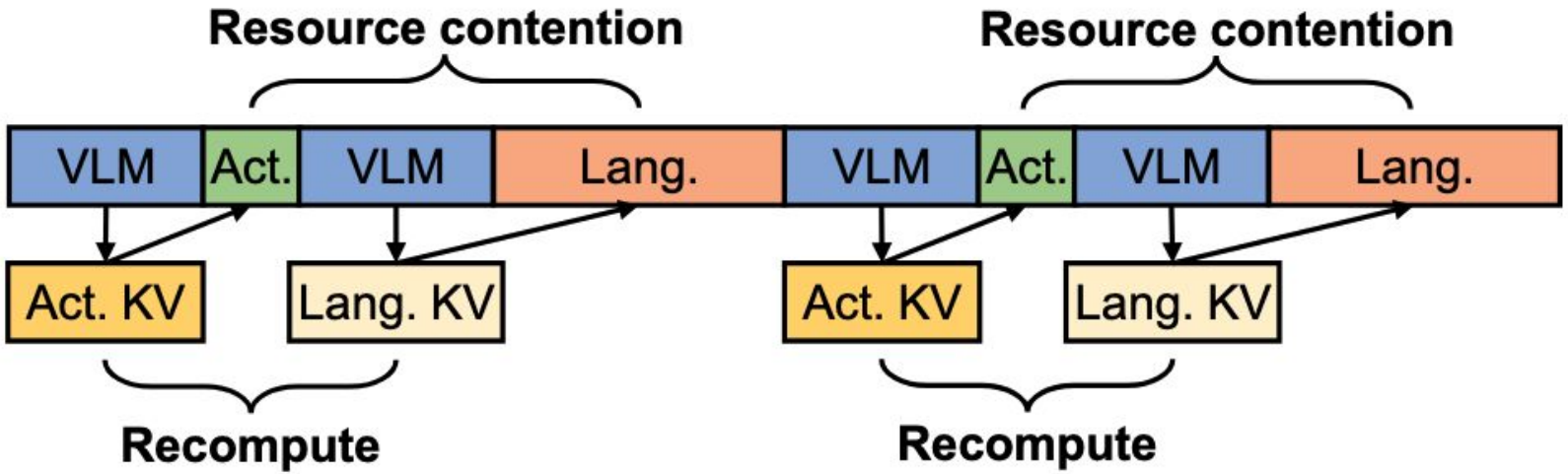
Mixture-of-Transformers VLAs

- **Route different outputs to modality-specific experts**
  - The backbone itself remains a language model, so one model can emit both actions and language



Architecture innovation ≠ Speed

- **Architectural multitasking does not translate into inference speedup**
  - Existing systems run each task as a separate forward pass -> Redundant computation
  - Even if the KV cache is shared, language decoding blocks action generation -> Resource contention



Previous Work: PagedAttention

- **Block-based KV management**
  - Fixed-size paging: Reduced memory fragmentation, achieving higher inference throughput.
  - Prefix caching: Enabled shared-prefix KV cache reuse, reducing redundant computation.

Request prompt: "Four score and seven years ago out"

Output: "fathers" → "brought" → …

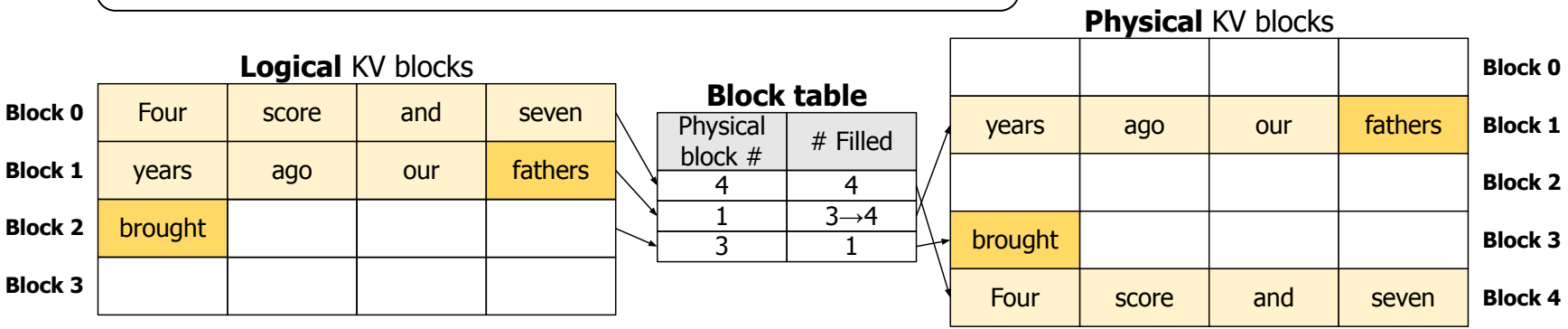
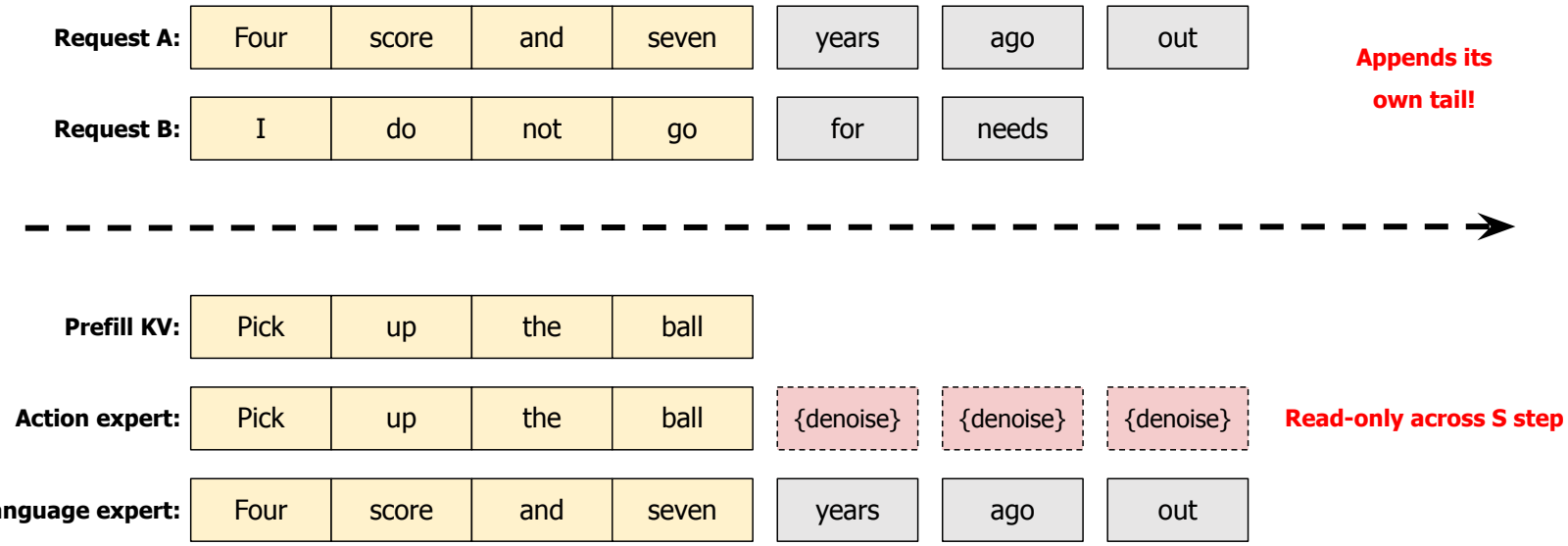


Fig. 5. Block table translation with PagedAttention

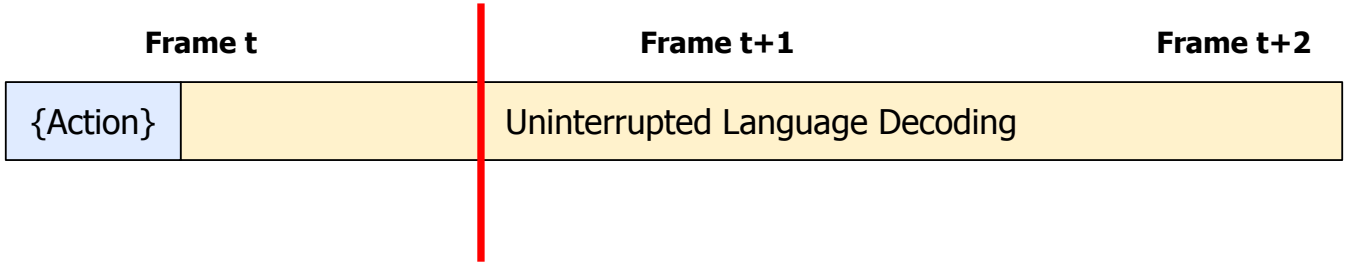
Why not using LLM-serving KV techniques

- **Two MoT-specific assumptions break standard LLM-serving KV techniques**
  - 1) Heterogeneous expert semantics:
    action reads the cache read-only over S denoising steps, language appends per token



Why not using LLM-serving KV techniques

- **Two MoT-specific assumptions break standard LLM-serving KV techniques**
  - 1) Frame-bounded decoding:
    continuous batching runs to completion, but robots need a per-frame step budget and resumption



The next action deadline passes while decoding is still running!!

Why not using LLM-serving KV techniques

- **Two MoT-specific assumptions break standard LLM-serving KV techniques**
  - 1) Frame-bounded decoding:

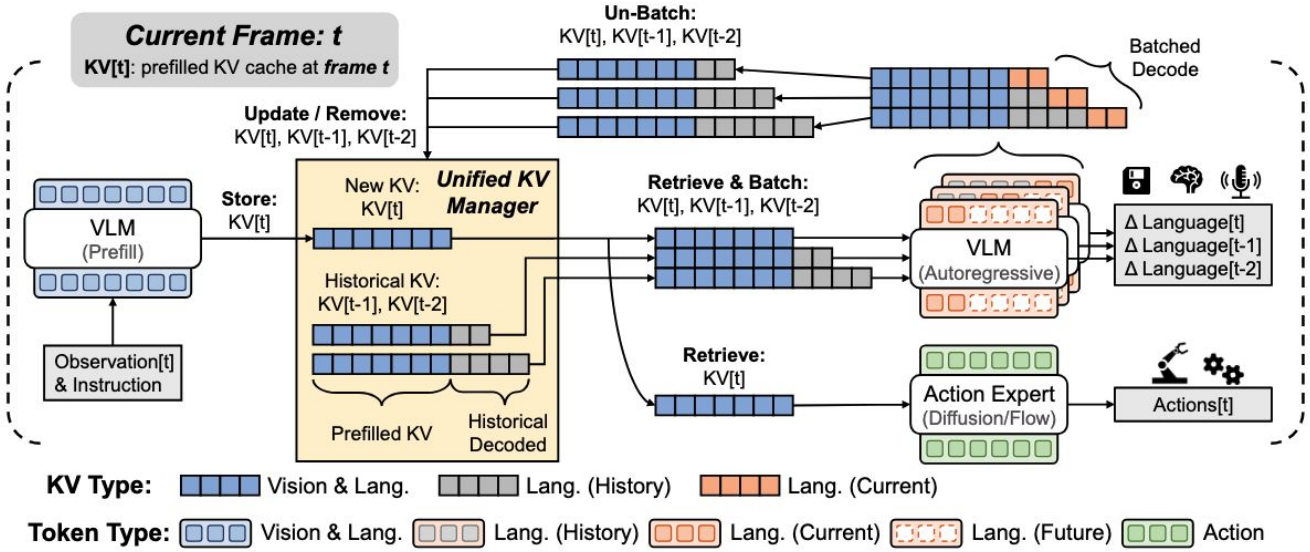
Maximize language throughput

Without ever missing the action deadline

The next action deadline passes while decoding is still running!!

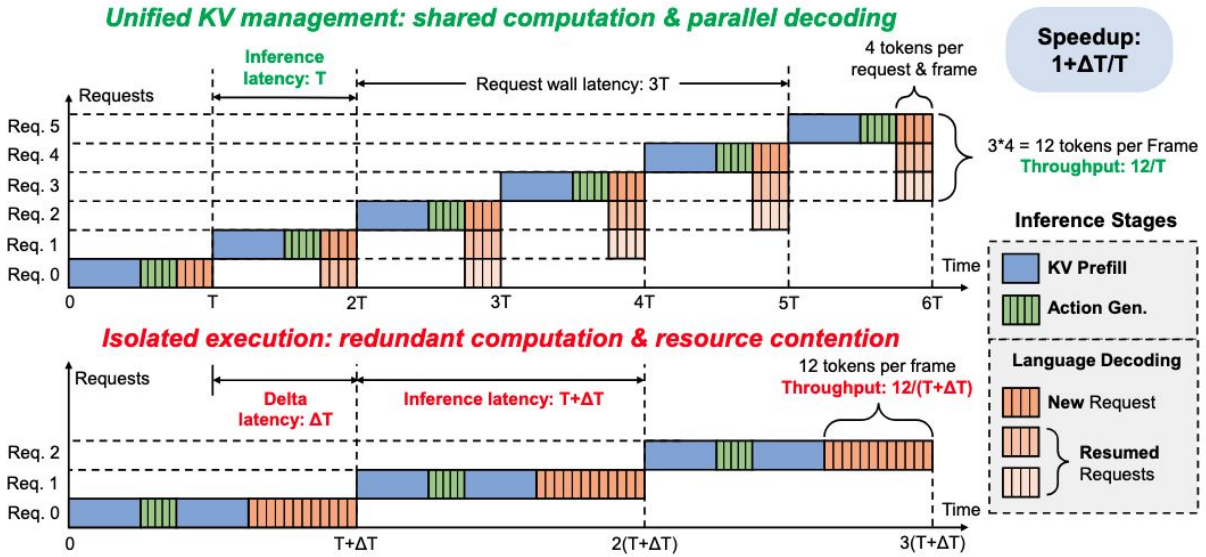
Cross-task KV sharing

- **Prefill the observation once, then fan the cache out**
  - Action: Reads K_t read-only over S steps + Releases it when the frame ends.
  - Language: Persists cache as a resumable state.



Cross-Frame Batching

- **In-flight requests born in different frames decode together**
  - Unfinished requests from past frames decode together.
  - Shorter frames → faster actions and more tokens.

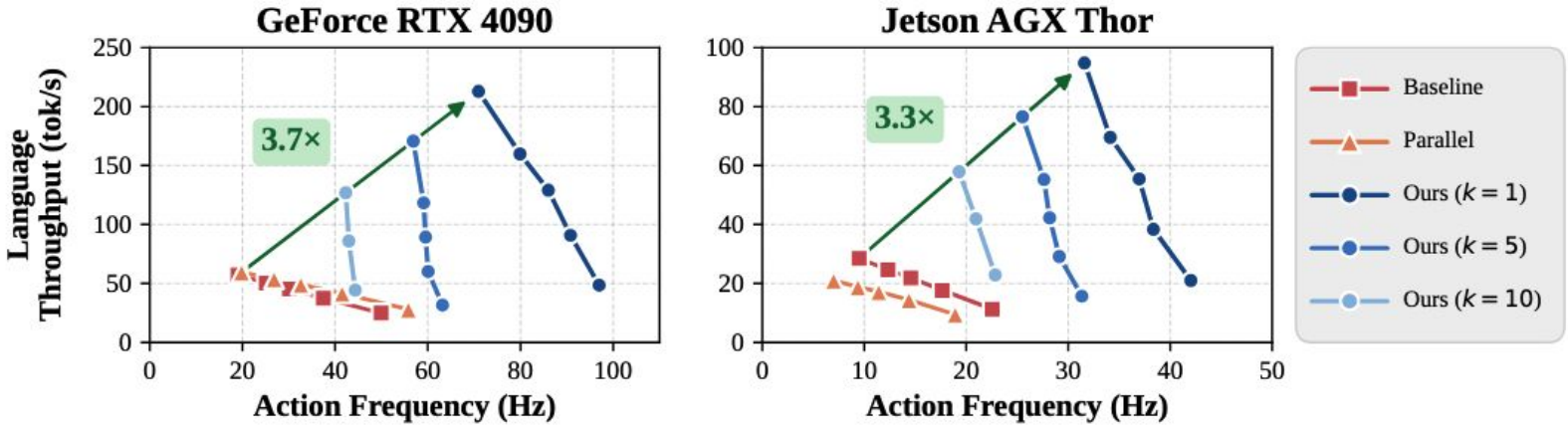


Evaluation Setup

- **Baseline**
  - Sequential isolated execution
  - Parallel isolated execution with MPS
- **Hardware**
  - RTX4090
  - Jetson AGX Thor
- **Metrics**
  - Action frequency(Hz)
  - Language throughput(tokens/s)
  - Average batch size
  - Memory(GB), Power(W), Energy/Request
- **Benchmark**
  - LIBERO
  - DROID
  - ALOHA

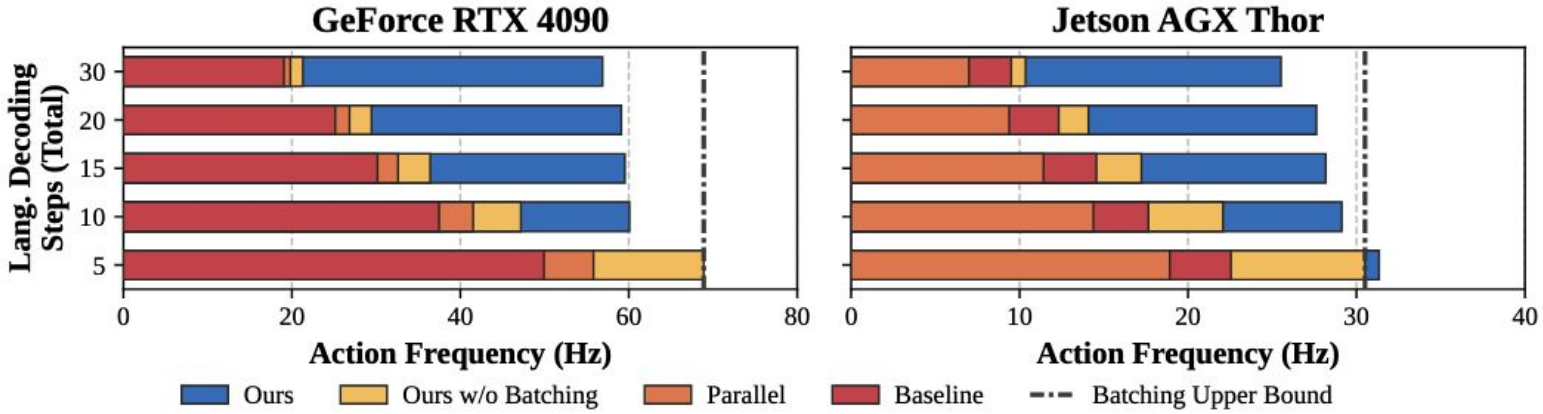
Result 1. Pareto Frontier

- **Action frequency & Language Throughput**
  - 1.2–3.7× on action frequency and throughput at once.



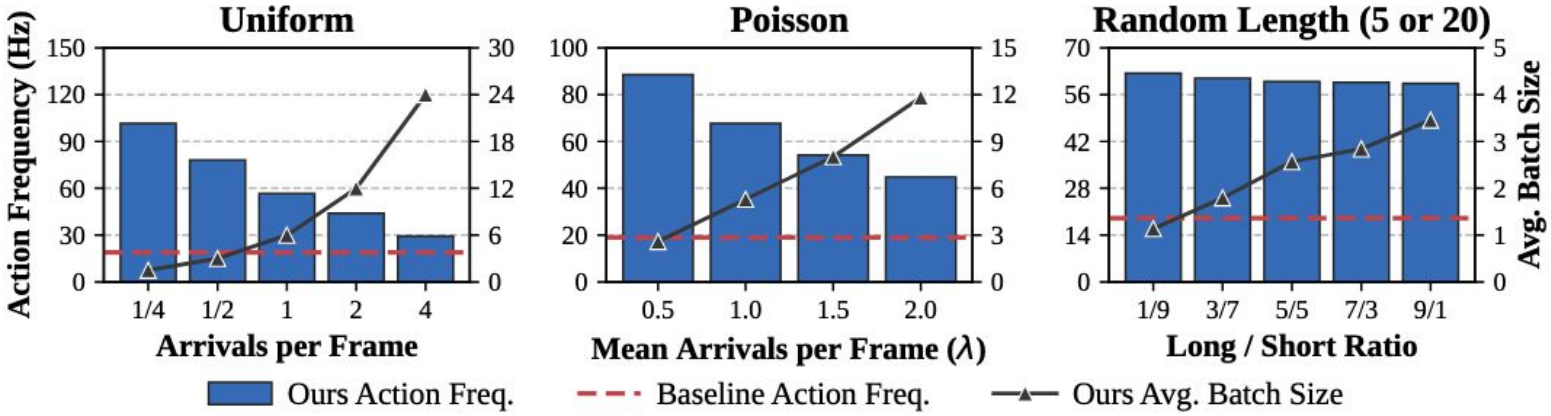
Result 2. Ablation

- **The two optimizations own different regimes**
  - KV sharing: 1.4× when decoding is short.
  - Batching: holds ~60 Hz as decoding grows, baseline falls to 19 Hz



Result 3. Workload Shift

- **Gains hold under uniform, Poisson, and mixed-length arrivals**
  - More arrivals → bigger batch, higher throughput
  - Per-request frequency drops, but total actions/s rises 4.4×



Conclusion

- **Recognize Challenge**
  - Identify multi-task parallelism as a target inference scenario for MoT VLAs.
- **Propose novel & unified design**
  - Propose unified KV cache management, treating the KV cache as a shared resource across tasks and over time.
- **Implement prototype on top of openpi**
  - Implement the design for π0.5, the official framework for a widely used MoT VLA.
- **Achieve superior on-device performance**
  - Achieve up to 3.7× speedup on RTX 4090 and Jetson AGX Thor, without degrading action quality.